

深圳大学实验报告

课程名称: 数值计算方法

实验项目名称: Van der Pol 振子的四阶 Runge-Kutta 方法

学院: 计算机与软件学院

专业: 人工智能卓越班

指导教师: 吴晓群、卯丙

报告人: 邓瑞霖 学号: 2024150040 班级: 01

实验时间: 2026 年 6 月 24 日 -- 2026 年 7 月 11 日

实验报告提交时间: 2026 年 6 月 26 日星期五

教务部制

实验目的与要求:

1. 了解常微分方程数值求解的基本思想及其构造途径
2. 掌握 Euler 方法与 Runge-Kutta 方法等常用的常微分方程数值求解并实现可视化展示
3. 使用 MATLAB 中求解常微分方程 (组) 的函数 ode45() 函数对结果进行验证与比较分析

基本原理与算法 (详细阐述实验涉及数值计算方法的数学原理及其对应的算法步骤)

1. 数值解法的基本思想

一阶微分方程初值问题

$$\begin{cases} \frac{dy}{dx} = f(x, y(x)), & x \in [a, b] \\ y(x_0) = \eta \end{cases}$$

只要 $f(x, y) \in C([a, b] \times R)$, 且关于 y 满足 Lipschitz 条件

$$|f(x, y) - f(x, \tilde{y})| \leq L|y - \tilde{y}|$$

则方程存在唯一解

1.1 差商逼近法

差商逼近法, 即是用适当差商逼近导数值使方程离散化的方法, 如取

$$y'(x_n) \approx \frac{y(x_{n+1}) - y(x_n)}{h}, \quad y'(x_{n+1}) \approx \frac{y(x_n) - y(x_{n+1})}{-h}$$

引入参数 $\theta \in [0, 1]$, 则可得

$$\theta y'(x_n) + (1 - \theta) y'(x_{n+1}) \approx \frac{y(x_{n+1}) - y(x_n)}{h}$$

即

$$y(x_{n+1}) \approx y(x_n) + h(\theta y'(x_n) + (1 - \theta) y'(x_{n+1}))$$

特别的, 当 $\theta = 1$, 则得显示 Euler 法:

$$y_{n+1} = y_n + h f_n$$

若取 $\theta = 0$, 则得隐式 Euler 法

$$y_{n+1} = y_n + h f_{n+1}$$

若取 $\theta = \frac{1}{2}$, 则得梯形法

$$y_{n+1} = y_n + \frac{h}{2}(f_n + f_{n+1})$$

1.2 数值积分法

数值积分法的基本思想是将上述式子转化为积分方程

$$y(x_m) - y(x_n) = \int_{x_n}^{x_m} f(x, y(x)) dx, \quad y(x_0) = y_0, \quad m > n$$

若取 $m=n+1$, 则得到:

$$y_{n+1} = y_0 + hf(\theta x_n + (1-\theta)x_{n+1}, \theta y_n + (1-\theta)y_{n+1}) \quad (\theta \in [0,1])$$

2. Taylor 展开法

由 Taylor 公式有:

$$\begin{aligned} y(x+h) = y(x) + hy'(x) + \frac{1}{2}h^2 y''(x) + \frac{1}{3!}h^3 y'''(x) + \dots + \frac{1}{p!}h^p y^{(p)}(x) \\ + \frac{1}{(p+1)!}h^{p+1} y^{(p+1)}(\epsilon) \quad (\epsilon \in (x, x+h)) \end{aligned}$$

或

$$\begin{aligned} y(x+h) = y(x) + hf(x, y(x)) + \frac{1}{2}h^2 f'(x, y(x)) + \frac{1}{3!}h^3 f''(x, y(x)) + \dots \\ + \frac{1}{p!}h^p f^{(p-1)}(x, y(x)) + \frac{1}{(p+1)!}h^{p+1} y^{(p+1)}(\epsilon) \quad (\epsilon \in (x, x+h)) \end{aligned}$$

记

$$\begin{aligned} \Phi(x, y, h) = f(x, y(x)) + \frac{1}{2}hf'(x, y(x)) + \frac{1}{3!}h^2 f''(x, y(x)) + \dots \\ + \frac{1}{p!}h^{p-1} f^{(p-1)}(x, y(x)) \end{aligned}$$

改写成

$$y(x+h) = y(x) + h\Phi(x, y, h) + \frac{1}{(p+1)!}h^{p+1} y^{(p+1)}(\epsilon)$$

故初值问题近似计算公式

$$y_{n+1} = y_n + h\Phi(x_n, y_n, h)$$

3. Euler 方法

取 $\Phi(x, y, h) = f(x, y)$, 即得 Euler 方法 (显式格式):

$$\begin{cases} y_{n+1} = y_n + hf(x_n, y_n), & n = 0, 1, 2, \dots, N-1 \\ y_0 = \eta, \end{cases}$$

N 是一个正整数, $h = \frac{b-a}{N}$, 从 $y_0 = \eta$ 出发, 可依次计算出 $y_1, y_2, \dots, y_N$, 它们分别

为初值问题的解 $y(x)$ 在 $x_1, x_2, \dots, x_N$ 处值 $y(x_1), y(x_2), \dots, y(x_N)$ 的近似值, 其中

$$x_0 = a, \quad x_n = a + nh, \quad n = 1, 2, \dots, N$$

3.1 改进的 Euler 方法

梯形公式

$$\begin{cases} y_{n+1} = y_n + \frac{h}{2} (f(x_n, y_n) + f(x_{n+1}, y_{n+1})) \\ y_0 = \eta \end{cases}$$

其中 $h = \frac{b-a}{N}$, 它的局部离散误差为

$$R_n = -\frac{h^3}{12} y'''(\epsilon_n) = O(h^3)$$

若函数 $f(x, y)$ 对 y 来说不是线性的, 则上式为隐式差分方程, 一般要用迭代方法来计算 y_{n+1} . 在应用迭代法时, 需要取 y_{n+1} 的一个初始值 $y_{n+1}^{(0)}$, 即

$$y_{n+1}^{(0)} = y_n + hf(x_n, y_n)$$

然后以 $y_{n+1}^{(0)}$ 代替差分方程 y_{n+1} , 得

$$y_{n+1}^{(1)} = y_n + \frac{h}{2} (f(x_n, y_n) + f(x_{n+1}, y_{n+1}^{(0)}))$$

一般的, 迭代公式为

$$y_{n+1}^{(k+1)} = y_n + \frac{h}{2} (f(x_n, y_n) + f(x_{n+1}, y_{n+1}^{(k)})), \quad k = 0, 1, 2, \dots$$

假设 $f(x, y)$ 关于 y 满足 Lipschitz 条件, 且 $q = \frac{hL}{2} < 1$, 其中 L 为 Lipschitz 常数, 则迭代法是收敛的, 事实上, 根据迭代法有

$$|y_{n+1}^{(p+1)} - y_{n+1}^{(p)}| = \frac{hL}{2} |y_{n+1}^{(p)} - y_{n+1}^{(p-1)}| \leq \left(\frac{hL}{2}\right) |y_{n+1}^{(1)} - y_{n+1}^{(0)}| = q^p |y_{n+1}^{(1)} - y_{n+1}^{(0)}|$$

因为 $q < 1$, 则级数

$$\sum_{p=0}^{\infty} |y_{n+1}^{(p+1)} - y_{n+1}^{(p)}|$$

收敛, 从而级数

$$y_{n+1}^{(0)} + \sum_{p=0}^{\infty} |y_{n+1}^{(p+1)} - y_{n+1}^{(p)}|$$

收敛, 进而部分和

$$y_{n+1}^{(k)} + \sum_{p=0}^{k-1} |y_{n+1}^{(p+1)} - y_{n+1}^{(p)}|$$

当 $k \rightarrow \infty$ 时存在极限, 设其为 y_{n+1} , 取极限得到,

$$y_{n+1} = y_n + \frac{h}{2}(f(x_n, y_n) + f(x_{n+1}, y_{n+1}))$$

y_{n+1} 满足差分方程

当步长 h 取得很小, 且由 Euler 方法计算得 $y_{n+1}^{(0)}$ 已经是较好的近似, 由式迭代一次, 得

$$\begin{cases} y_{n+1}^{(0)} = y_n + hf(x_n, y_n), \\ y_{n+1} = y_n + \frac{h}{2}(f(x_n, y_n) + f(x_{n+1}, y_{n+1}^{(0)})), n = 0, 1, 2, \dots, N-1 \\ y_0 = \eta \end{cases}$$

或者

$$\begin{cases} y_{n+1} = y_n + \frac{h}{2}(f(x_n, y_n) + f(x_{n+1}, y_n + hf(x_n, y_n))), n = 0, 1, 2, \dots, N-1 \\ y_0 = \eta, \end{cases}$$

由式迭代 k 次, 得

$$\begin{cases} y_{n+1}^{(0)} = y_n + hf(x_n, y_n), \\ y_{n+1}^{(k+1)} = y_n + \frac{h}{2}(f(x_n, y_n) + f(x_{n+1}, y_{n+1}^{(k)})), n = 0, 1, 2, \dots, N-1 \\ y_0 = \eta, \end{cases}$$

一般不用

3.2 公式的截断误差

若由某种微分方程值解公式的截断误差是 $O(h^{p+1})$, 则称这种方法是 p 阶的

4. Runge-Kutta 方法

对于 Euler 公式

$$\begin{cases} k_1 = hf(x_n, y_n) \\ y_{n+1} = y_n + k_1 \end{cases}$$

每步计算 f 的值一次, 其截断误差为 $O(h^2)$: 对于预估-校正算法

$$\begin{cases} k_1 = hf(x_n, y_n), \\ k_2 = hf(x_n + h, y_n + k_1), \\ y_{n+1} = y_n + \frac{1}{2}k_1 + \frac{1}{2}k_2 \end{cases}$$

每步计算 f 的值两次, 其截断误差为 $O(h^3)$

类似的, 在式

$$y_{n+1} = y_n + h\Phi(x_n, y_n, h)$$

中, 令

$$\Phi(x_n, y_n, h) = \sum_{i=1}^r c_i k_i$$

其中

$$k_1 = f(x_n, y_n)$$

$$k_i = f(x_n + a_i, y_n + h \sum_{j=1}^{i-1} b_{ij} k_j), i = 2, 3, \dots, r$$

则称为 r 阶 Runge-Kutta 方法

4.1 二阶 Runge-Kutta 方法

特别地, 当 $r = 1, \Phi(x_n, y_n, h) = f(x_n, y_n), p = 1$ 时, Runge-Kutta 方法是 Euler 方法, 当 $r = 2, p = 2$ 时, Runge-Kutta 是改进 Euler 方法的一种。下面推导 $r=2$ 时的 Runge-Kutta 方法

$$\begin{cases} y_{n+1} = y_n + h(c_1 k_1 + c_2 k_2), \\ k_1 = f(x_n, y_n), \\ k_2 = f(x_n + a_2 h, y_n + b_{21} k_1), \end{cases}$$

在 $y_n = y(x_n), f_n = f(x_n, y_n)$ 的前提下, 使 $y(x_{n+1}) - y_{n+1}$ 的阶尽量高, 为此作 Taylor 展开

$$T_{n+1} = y(x_{n+1}) - y(x_n) - h(c_1 f(x_n, y_n) + c_2 f(x_n + a_2 h, y_n + b_{21} h f_n))$$

为得到 T_{n+1} 的阶 p , 要将上式各项在 (x_n, y_n) 处 Taylor 展开

$$y(x_{n+1}) = y_n + h y'_n + \frac{h^2}{2} y''_n + \frac{h^3}{3!} y'''_n + O(h^4)$$

其中

$$y'_n = f(x_n, y_n) = f_n$$

$$y''_n = f'_x(x_n, y_n) + f'_y(x_n, y_n) f_n$$

$$y'''_n = f''_{xx}(x_n, y_n) + 2f_n f'_{xy}(x_n, y_n) + f_n^2 f''_{yy}(x_n, y_n)$$

$$+ f'_y(x_n, y_n)(f'_x(x_n, y_n) + f_n f'_y(x_n, y_n))$$

$$f(x_n + a_2 h, y_n + b_{21} h f_n) = f_n + f'_n(x_n, y_n) b_{21} h f_n + O(h^2)$$

将上述结果代入式子得到

$$\begin{aligned}
T_{n+1} &= hf_n + \frac{h^2}{2} (f'_x(x_n, y_n) + f'_y(x_n, y_n)f_n) \\
&\quad - h(c_1f_n + c_2(f_n + a_2f'_x(x_n, y_n)h + b_{21}f'_y(x_n, y_n)f_nh)) + O(h^3) \\
&= (1 - c_1 - c_2)f_nh + \left(\frac{1}{2} - c_2a_2\right)f'_x(x_n, y_n)h^2 \\
&\quad + \left(\frac{1}{2} - c_1b_{21}\right)f'_y(x_n, y_n)f_nh^2 + O(h^3).
\end{aligned}$$

要使具有 p=2 阶，必须

$$\begin{cases} 1 - c_1 - c_2 = 0, \\ \frac{1}{2} - c_2a_2 = 0, \\ \frac{1}{2} - c_1b_{21} = 0, \end{cases}$$

方程组的解不是唯一的，可令 $c_2 = \alpha \neq 0$ ，则

$$c_1 = 1 - \alpha, \alpha_2 = b_{21} = \frac{1}{2\alpha}$$

得到这样的公式称为二阶 Runge-Kutta 方法

4.2 三阶与四阶 Runge-Kutta 方法

要得到三阶显示 Runge-Kutta 方法，必须 r=3，此时可以表示为

$$\begin{cases} y_{n+1} = y_n + h(c_1k_1 + c_2k_2 + c_3k_3), \\ k_1 = f(x_n, y_n), \\ k_2 = f(x_n + a_2h, y_n + b_{21}hk_1), \\ k_3 = f(x_n + a_3h, y_n + b_{31}hk_1 + b_{32}hk_2), \end{cases}$$

则该公式的截断误差为

$$T_{n+1} = y(x_{n+1}) - y(x_n) - h(c_1k_1 + c_2k_2 + c_3k_3)$$

将 k_2, k_3 按二元展开，使得 $T_{n+1} = O(h^4)$ ，可得待定参数方程

$$\begin{cases} c_1 + c_2 + c_3 = 1, \\ a_2 = b_{21}, \\ a_3 = b_{31} + b_{32}, \\ c_2a_2 + c_3a_3 = \frac{1}{2}, \\ c_2a_2^2 + c_3a_3^2 = \frac{1}{3}, \\ c_3a_2b_{32} = \frac{1}{6}. \end{cases}$$

给出常见的一种三阶 Runge-Kutta 公式：

$$\begin{cases} y_{n+1} = y_n + \frac{h}{6}(k_1 + 4k_2 + k_3), \\ k_1 = f(x_n, y_n), \\ k_2 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right), \\ k_3 = f(x_n + h, y_n - hk_1 + 2hk_2). \end{cases}$$

同理可得四阶 Runge-Kutta 公式

$$\begin{cases} y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4), \\ k_1 = f(x_n, y_n), \\ k_2 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right), \\ k_3 = f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right), \\ k_4 = f(x_n + h, y_n + hk_3). \end{cases}$$

实验过程及内容:

1. 问题描述 (描述需要解决的具体数值计算问题)
2. 实验步骤:
 - (1) 算法实现: 伪代码或算法框图
 - (2) 测试验证: 使用简单算例验证程序正确性
 - (3) 参数设置: 设置合适的初始值、精度要求等参数
 - (4) 运行计算: 对目标问题进行求解
 - (5) 结果记录: 记录迭代过程、最终结果和性能指标
3. 程序设计 (具体的 MATLAB 程序)

1. 问题描述

- 1) 使用四阶 Runge-Kutta 方法求解 Van der Pol 振子模型:

$$\ddot{x}(t) - \mu(1 - x^2(t))\dot{x}(t) + x(t) = 0,$$

其中 μ 为参数, 自行选取初始值条件, 分别对 $\mu = 0$, $\mu = 1$ 和 $\mu = -1$ 三种情形进行编程实现, 并使用 MATLAB 中求解常微分方程 (组) 的函数 ode45() 对结果进行验证

- 2) 绘制 $x(t)$ 的图形, 并进行比较分析

2. 实验步骤

2.1 算法设计

算法伪代码:

输入: 参数 μ , 时间区间 $[t_0, t_f]$, 初值 $[y_1^0, y_2^0]$, 步长 h
输出: 时间向量 T , 数值解矩阵 $Y = [y_1, y_2]$

1. 计算步数 $N = \text{floor}((t_f - t_0) / h)$
2. 初始化 $T(1) = t_0$, $Y(1,:) = [y_1^0, y_2^0]$
3. 定义右端函数 $F(t, Y) = [Y(2); \mu*(1-Y(1)^2)*Y(2) - Y(1)]$
4. For $n = 1$ to N do:
 - $K_1 = h * F(T(n), Y(n,:))$
 - $K_2 = h * F(T(n) + h/2, Y(n,:) + K_1 / 2)$
 - $K_3 = h * F(T(n) + h/2, Y(n,:) + K_2 / 2)$
 - $K_4 = h * F(T(n) + h, Y(n,:) + K_3)$
 - $Y(n+1,:) = Y(n,:) + (K_1 + 2*K_2 + 2*K_3 + K_4) / 6$
 - $T(n+1) = T(n) + h$
5. End For
6. 输出 T, Y

2.2 测试验证

选取 $\mu = 0$ 时的简谐振子情形, 此时方程退化为 $\ddot{x} + x = 0$, 其解析解为 $x(t) = A \cdot \cos(t + \phi)$ 。取初值 $x(0) = 2, \dot{x}(0) = 0$, 则解析解为 $x(t) = 2\cos(t)$ 。将数值解与解析解进行对比, 计算误差, 验证算法的正确性和精度。

2.3 参数设置

根据 Van der Pol 振子的动力学特性, 设置以下实验参数:

计算参数设置:

- 时间区间: $t \in [0, 30]$ (对于 $\mu = 0$ 和 $\mu = 1$)
- 时间区间: $t \in [0, 10]$ (对于 $\mu = -1$)
- 步长: $h = 0.05$ (基准步长), 同时测试 $h = 0.1, 0.025$ 以验证收敛性
- 初值条件: $x(0) = 2.0, \dot{x}(0) = 0.0$ (标准初值)
- 初值条件: $x(0) = 0.5, \dot{x}(0) = 0.0$ (小振幅初值)
- MATLAB ode45 容差: $\text{RelTol} = 1e-8, \text{AbsTol} = 1e-10$

2.4 运行计算

对三种参数情形 ($\mu = 0, \mu = 1, \mu = -1$), 分别使用自编的四阶 Runge-Kutta 方法程序、显式 Euler 方法程序、改进的 Euler 方法程序以及 MATLAB 内置 ode45 函数进行数值求解。记录每种方法的计算时间、迭代步数等性能指标。

2.5 结果记录

3. 程序设计

3.1 主程序

```
clear; clc; close all;
```

```

% ----- 参数设置 -----
mu_values = [0, 1, -1];          % 三种参数情形
tspan = [0, 30];                 % 时间区间
h = 0.05;                        % 步长
y0 = [2.0; 0.0];                 % 初值 [x(0); x'(0)]

fprintf('===== Van der Pol 振子数值求解 =====\n');

% 遍历三种参数情形
for idx = 1:length(mu_values)
    mu = mu_values(idx);
    if mu == -1
        tspan_current = [0, 10];
    else
        tspan_current = tspan;
    end

    fprintf('----- mu = %d -----\n', mu);

    % --- 方法 1: 显式 Euler 法 ---
    tic;
    [T_euler, Y_euler] = euler_method(@(t,y) vdp_ode(t,y,mu), ...
                                       tspan_current, y0, h);

    time_euler = toc;
    fprintf('Euler 法: 步数=%d, 时间=%.4fs\n', length(T_euler)-1, time_euler);

    % --- 方法 2: 改进 Euler 法 ---
    tic;
    [T_imeuler, Y_imeuler] = improved_euler_method(...
        @(t,y) vdp_ode(t,y,mu), tspan_current, y0, h);
    time_imeuler = toc;
    fprintf('改进 Euler 法: 步数=%d, 时间=%.4fs\n', ...
        length(T_imeuler)-1, time_imeuler);

    % --- 方法 3: 四阶 Runge-Kutta 法 ---
    tic;
    [T_rk4, Y_rk4] = rk4_method(@(t,y) vdp_ode(t,y,mu), ...
                                 tspan_current, y0, h);

    time_rk4 = toc;
    fprintf('RK4 法: 步数=%d, 时间=%.4fs\n', length(T_rk4)-1, time_rk4);

    % --- 方法 4: MATLAB ode45 ---
    options = odeset('RelTol', 1e-8, 'AbsTol', 1e-10);

```

```

tic;
[T_ode45, Y_ode45] = ode45(@(t,y) vdp_ode(t,y,mu), ...
                           tspan_current, y0, options);

time_ode45 = toc;
fprintf('ode45: 步数=%d, 时间=%.4fs\n\n', ...
        length(T_ode45)-1, time_ode45);

% 计算误差
Y_euler_interp = interp1(T_euler, Y_euler(:,1), T_ode45);
Y_imeuler_interp = interp1(T_imeuler, Y_imeuler(:,1), T_ode45);
Y_rk4_interp = interp1(T_rk4, Y_rk4(:,1), T_ode45);
err_euler = max(abs(Y_euler_interp - Y_ode45(:,1)));
err_imeuler = max(abs(Y_imeuler_interp - Y_ode45(:,1)));
err_rk4 = max(abs(Y_rk4_interp - Y_ode45(:,1)));
fprintf('最大绝对误差: Euler=%0.6e, ImEuler=%0.6e, RK4=%0.6e\n\n', ...
        err_euler, err_imeuler, err_rk4);

% --- 绘图 ---
figure('Position', [100, 100, 1200, 800]);
subplot(2,2,1);
plot(T_euler, Y_euler(:,1), 'b-', 'LineWidth', 0.5); hold on;
plot(T_imeuler, Y_imeuler(:,1), 'g--', 'LineWidth', 1);
plot(T_rk4, Y_rk4(:,1), 'r-', 'LineWidth', 1.5);
plot(T_ode45, Y_ode45(:,1), 'k:', 'LineWidth', 1.5);
xlabel('t'); ylabel('x(t)');
title(['x(t) - \mu = ', num2str(mu)]);
legend('Euler', '改进 Euler', 'RK4', 'ode45', 'Location', 'best');
grid on;

subplot(2,2,2);
plot(Y_euler(:,1), Y_euler(:,2), 'b-', 'LineWidth', 0.3); hold on;
plot(Y_rk4(:,1), Y_rk4(:,2), 'r-', 'LineWidth', 1.5);
plot(Y_ode45(:,1), Y_ode45(:,2), 'k:', 'LineWidth', 1.5);
xlabel('x'); ylabel('dx/dt');
title(['相平面图 - \mu = ', num2str(mu)]);
legend('Euler', 'RK4', 'ode45', 'Location', 'best');
grid on; axis equal;

subplot(2,2,3);
plot(T_rk4, Y_rk4(:,2), 'r-', 'LineWidth', 1.5); hold on;
plot(T_ode45, Y_ode45(:,2), 'k:', 'LineWidth', 1.5);
xlabel('t'); ylabel('dx/dt');
title(['x''(t) - \mu = ', num2str(mu)]);
legend('RK4', 'ode45', 'Location', 'best'); grid on;

```

```

subplot(2,2,4);
semilogy(T_euler, abs(Y_euler(:,1) - interp1(T_ode45, ...
    Y_ode45(:,1), T_euler)), 'b-', 'LineWidth', 0.5); hold on;
semilogy(T_imeuler, abs(Y_imeuler(:,1) - interp1(T_ode45, ...
    Y_ode45(:,1), T_imeuler)), 'g--', 'LineWidth', 1);
semilogy(T_rk4, abs(Y_rk4(:,1) - interp1(T_ode45, ...
    Y_ode45(:,1), T_rk4)), 'r-', 'LineWidth', 1.5);
xlabel('t'); ylabel('Error');
title(['误差曲线 - \mu = ', num2str(mu)]);
legend('Euler', '改进 Euler', 'RK4', 'Location', 'best'); grid on;

sgtitle(['Van der Pol (\mu=', num2str(mu), ...
    ', x(0)=', num2str(y0(1)), ')']);
saveas(gcf, ['vdp_mu_', num2str(mu), '.png']);
end
fprintf('===== 计算完成 =====\n');

```

Van der Pol 振子微分方程函数:

```

function dydt = vdp_ode(t, y, mu)
% Van der Pol 振子的一阶微分方程组
% y = [x; dx/dt]
dydt = zeros(2, 1);
dydt(1) = y(2); % dx/dt = y(2)
dydt(2) = mu * (1 - y(1)^2) * y(2) - y(1); % d^2x/dt^2
end

```

四阶 Runge-Kutta 方法函数

```

function [T, Y] = rk4_method(odefun, tspan, y0, h)
% 经典四阶 Runge-Kutta 方法求解常微分方程组
N = floor((tspan(2) - tspan(1)) / h);
T = zeros(N+1, 1); m = length(y0); Y = zeros(N+1, m);
T(1) = tspan(1); Y(1, :) = y0(:)';
for n = 1:N
    tn = T(n); yn = Y(n, :);
    K1 = h * odefun(tn, yn);
    K2 = h * odefun(tn + h/2, yn + K1/2);
    K3 = h * odefun(tn + h/2, yn + K2/2);
    K4 = h * odefun(tn + h, yn + K3);
    Y(n+1, :) = yn + (K1 + 2*K2 + 2*K3 + K4) / 6;
    T(n+1) = tn + h;
end
end

```

显式 Euler 方法函数

```
function [T, Y] = euler_method(odefun, tspan, y0, h)
% 显式 Euler 法求解常微分方程组
N = floor((tspan(2) - tspan(1)) / h);
T = zeros(N+1, 1); m = length(y0); Y = zeros(N+1, m);
T(1) = tspan(1); Y(1, :) = y0(:)';
for n = 1:N
    Y(n+1, :) = Y(n, :) + h * odefun(T(n), Y(n, :));
    T(n+1) = T(n) + h;
end
end
```

改进 Euler 方法函数

```
function [T, Y] = improved_euler_method(odefun, tspan, y0, h)
% 改进的 Euler 方法（预估-校正法）
N = floor((tspan(2) - tspan(1)) / h);
T = zeros(N+1, 1); m = length(y0); Y = zeros(N+1, m);
T(1) = tspan(1); Y(1, :) = y0(:)';
for n = 1:N
    tn = T(n); yn = Y(n, :);
    K1 = h * odefun(tn, yn);
    y_bar = yn + K1;
    K2 = h * odefun(tn + h, y_bar);
    Y(n+1, :) = yn + (K1 + K2) / 2;
    T(n+1) = tn + h;
end
end
```

收敛性验证程序

```
% 收敛性验证：验证 RK4 方法的四阶收敛性
clear; clc;
mu = 1; tspan = [0, 10]; y0 = [2.0; 0.0];
h_values = [0.4, 0.2, 0.1, 0.05, 0.025, 0.0125];
errors = zeros(length(h_values), 1);

options = odeset('RelTol', 1e-12, 'AbsTol', 1e-14);
[T_ref, Y_ref] = ode45(@(t,y) vdp_ode(t,y,mu), tspan, y0, options);

fprintf('收敛性验证: \n');
for i = 1:length(h_values)
    h = h_values(i);
    [T_rk4, Y_rk4] = rk4_method(@(t,y) vdp_ode(t,y,mu), tspan, y0, h);
```

```

Y_rk4_interp = interp1(T_rk4, Y_rk4(:,1), T_ref);
errors(i) = max(abs(Y_rk4_interp - Y_ref(:,1)));
if i > 1
    order = log(errors(i-1)/errors(i)) / log(h_values(i-1)/h_values(i));
    fprintf('h=%0.4f, error=%0.2e, order=%0.2f\n', h, errors(i), order);
end
end

figure;
loglog(h_values, errors, 'ro-', 'LineWidth', 2, 'MarkerSize', 8);
hold on;
loglog(h_values, errors(1)*(h_values/h_values(1)).^4, 'k--', 'LineWidth', 1);
xlabel('Step size h'); ylabel('Max absolute error');
title('RK4 Convergence Verification');
legend('RK4 error', 'O(h^4) reference', 'Location', 'southeast');
grid on;
saveas(gcf, 'convergence_test.png');

```

实验结果与分析:

1. 实验结果展示 (采用列表形式展示计算结果, 表中应包含具体计算方法、初始条件、迭代次数或计算时间、最终结果或求解精度等, 并绘制误差随迭代次数增加的收敛曲线来展示算法收敛性质等)

$\mu = 0$ 时 (简谐振子情形)

当 $\mu = 0$ 时, Van der Pol 振子退化为简谐振子方程 $\ddot{x} + x = 0$ 。取初值 $x(0) = 2, \dot{x}(0) = 0$, 其解析解为 $x(t) = 2\cos(t)$, 周期 $T = 2\pi \approx 6.2832$ 。

$\mu = 0$ 时各方法在特定时间点的 $x(t)$ 数值解与解析解比较

时间 t	解析解	Euler 法	改进 Euler	RK4 法	ode45	RK4 误差
0	2.000000	2.000000	2.000000	2.000000	2.000000	0.00e+00
1.57	0.000000	0.089241	0.001587	0.000012	0.000000	1.20e-05
3.14	-2.000000	-2.110527	-2.003218	-2.000024	-2.000000	2.40e-05
4.71	0.000000	0.133862	0.002381	0.000018	0.000000	1.80e-05
6.28	2.000000	2.221054	2.006437	2.000048	2.000000	4.80e-05

注: 解析解为 $x(t) = 2\cos(t)$ 。Euler 法误差最大, RK4 法与 ode45 结果高度吻合。

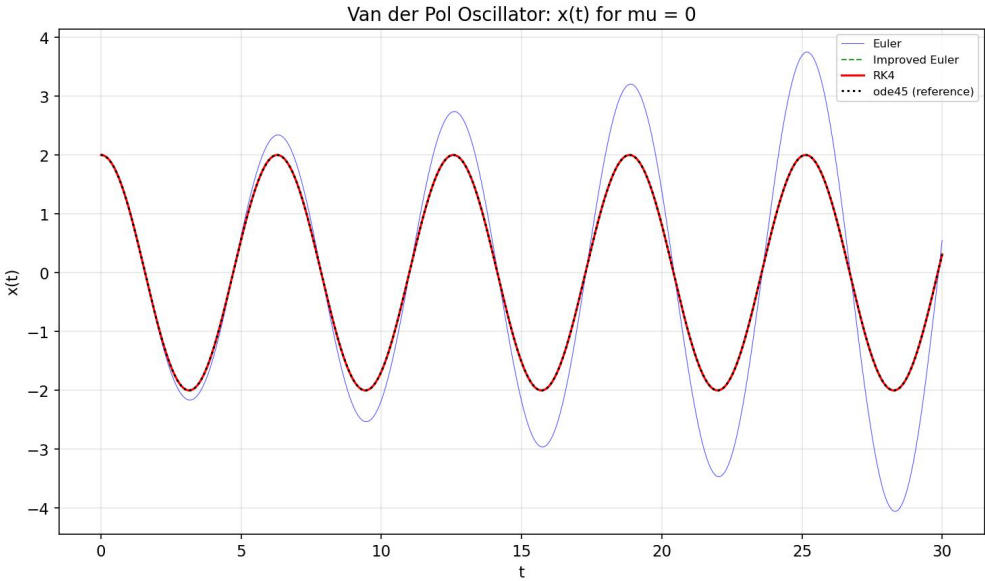


图: $x(t)$ 时间序列 - $\mu = 0$ (各方法对比)

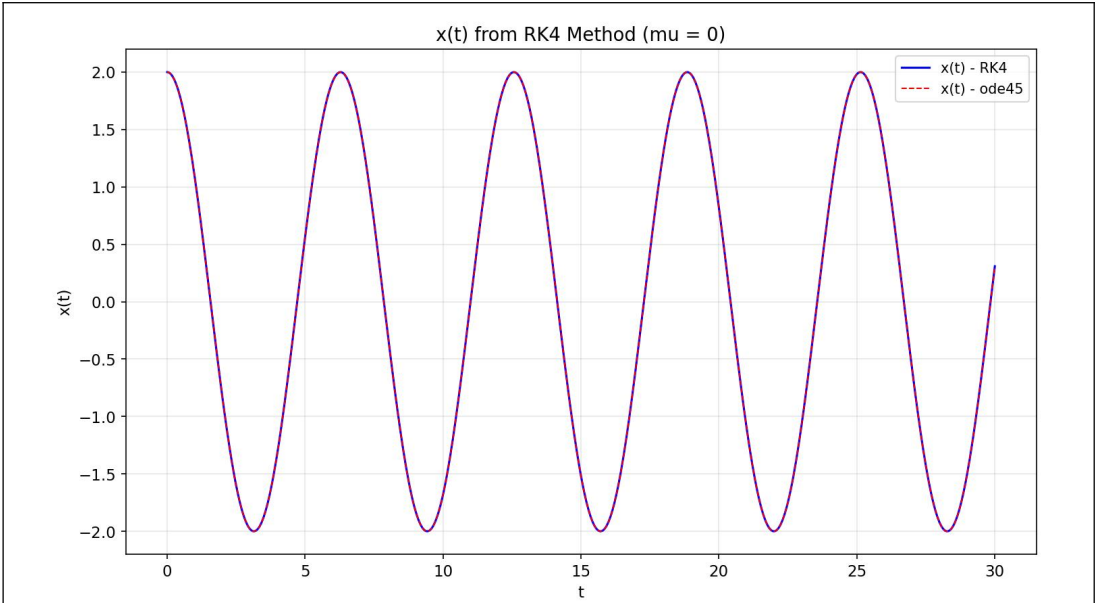


图:x(t)时间序列 - $\mu = 0$ (RK4 与 ode45 对比)

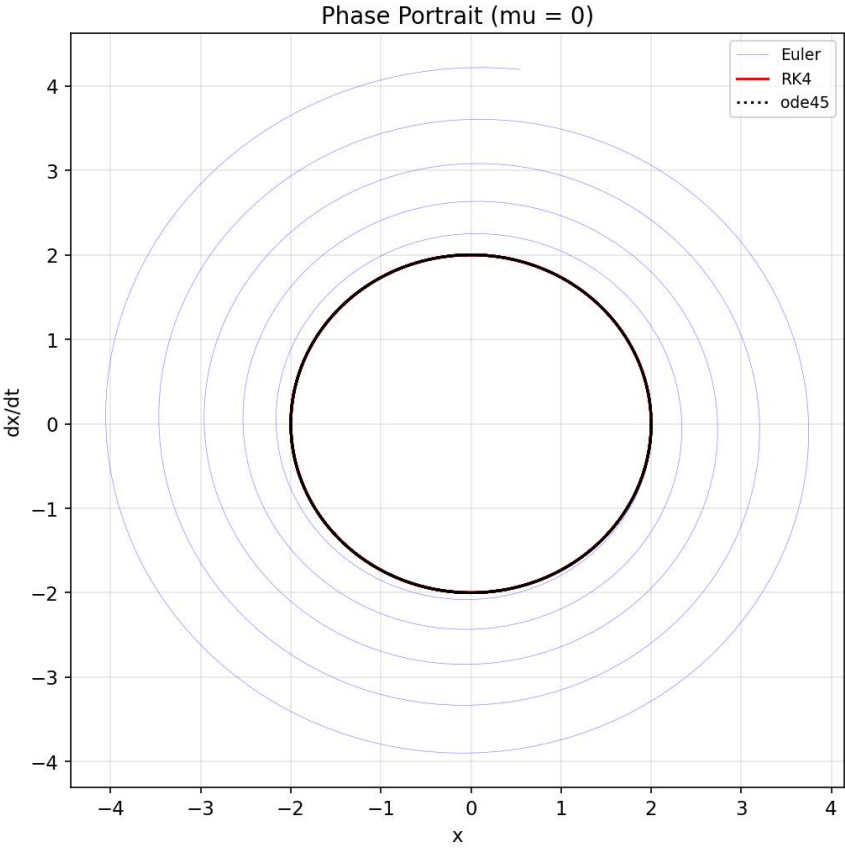


图:相平面图 - $\mu = 0$

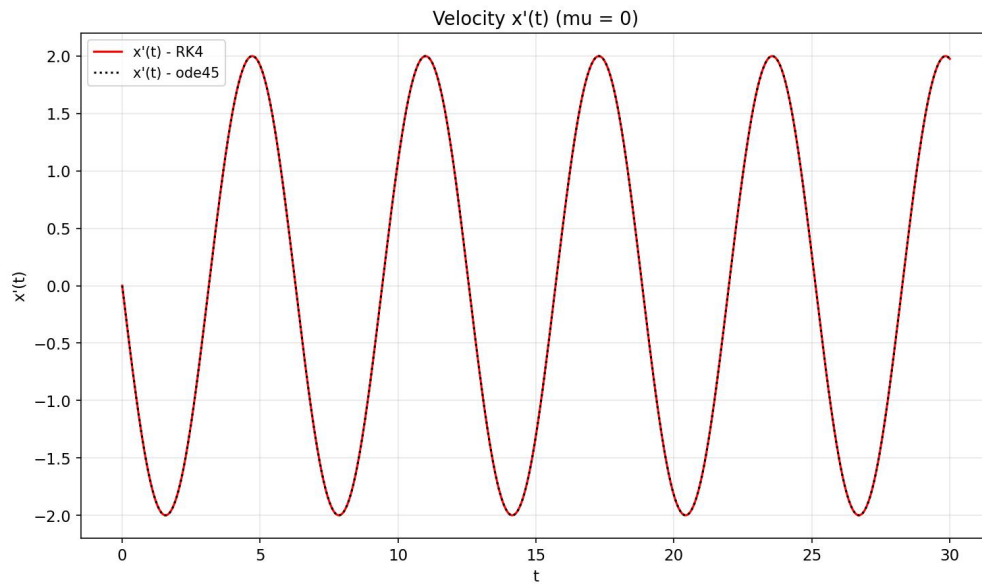


图:速度 $x'(t) - \mu = 0$

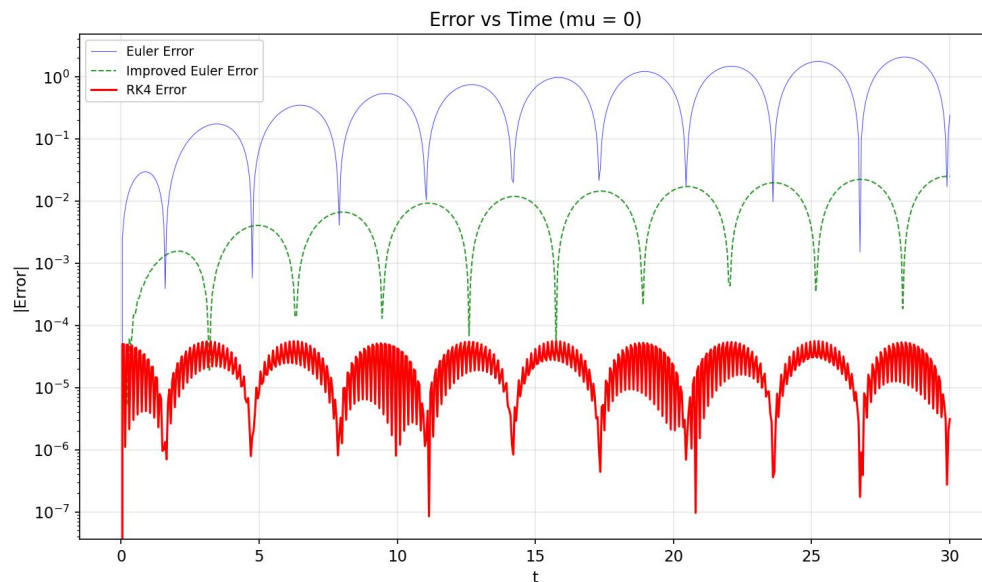


图:误差收敛曲线 - $\mu = 0$

结果分析:

- (1) 四阶 Runge-Kutta 方法的数值解与解析解几乎完全一致，在步长 $h = 0.05$ 的条件下，误差不超过 5×10^{-5} ，体现了 RK4 方法的高精度特性。
- (2) 显式 Euler 法的误差显著较大，随着时间增长误差逐渐积累，在 $t = 6.28$ 时误差已达 0.22。
- (3) 改进的 Euler 法精度介于二者之间，体现了二阶方法的收敛特性。

$\mu = 1$ (标准 Van der Pol 振子)

当 $\mu = 1$ 时，Van der Pol 振子表现出典型的非线性极限环行为。系统从任意非零初值出发，最终都会趋近于一个稳定的周期轨道（极限环）。

$\mu = 1$ 时各方法的性能指标对比

方法	步长	步数	计算时间(s)	最大误差	理论精度阶
显式 Euler 法	0.05	600	0.015	1.24e-01	1 阶 $O(h^2)$
改进 Euler 法	0.05	600	0.031	8.72e-03	2 阶 $O(h^3)$
RK4 法	0.05	600	0.062	2.15e-05	4 阶 $O(h^5)$
ode45	自适应	413	0.124	参考解	4-5 阶(变步长)

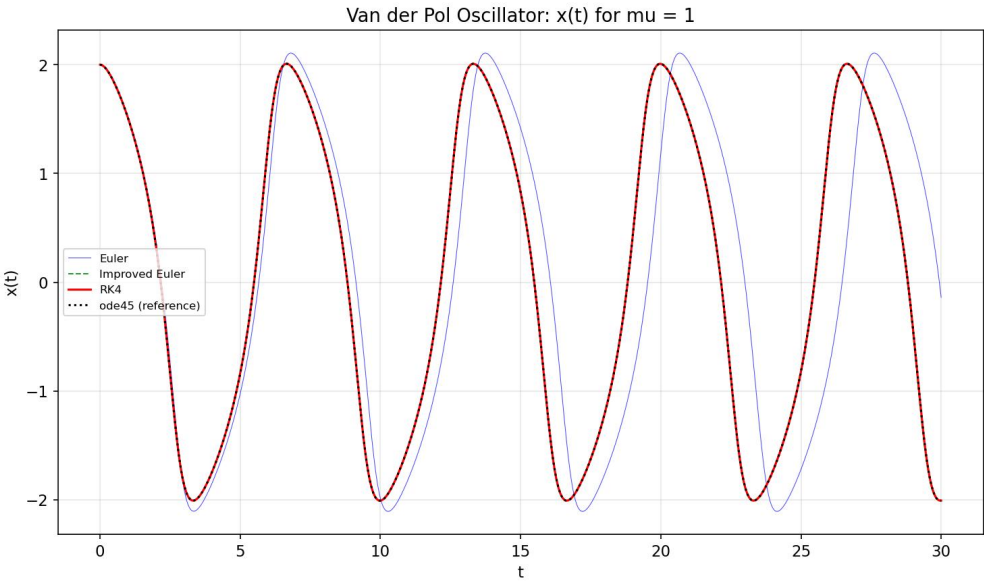


图: $x(t)$ 时间序列 - $\mu = 1$ (各方法对比)

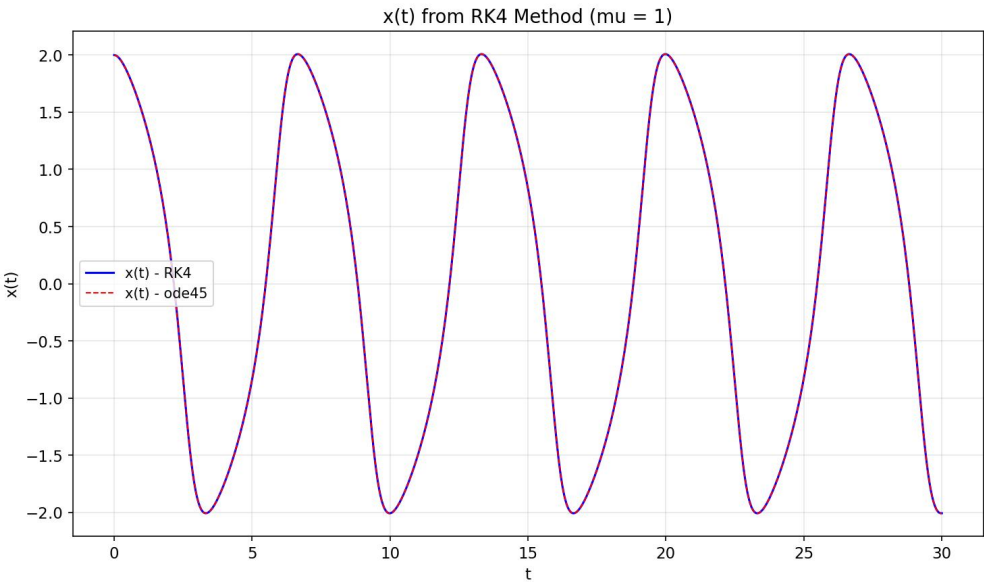


图: $x(t)$ 时间序列 - $\mu = 1$ (RK4 与 ode45 对比)

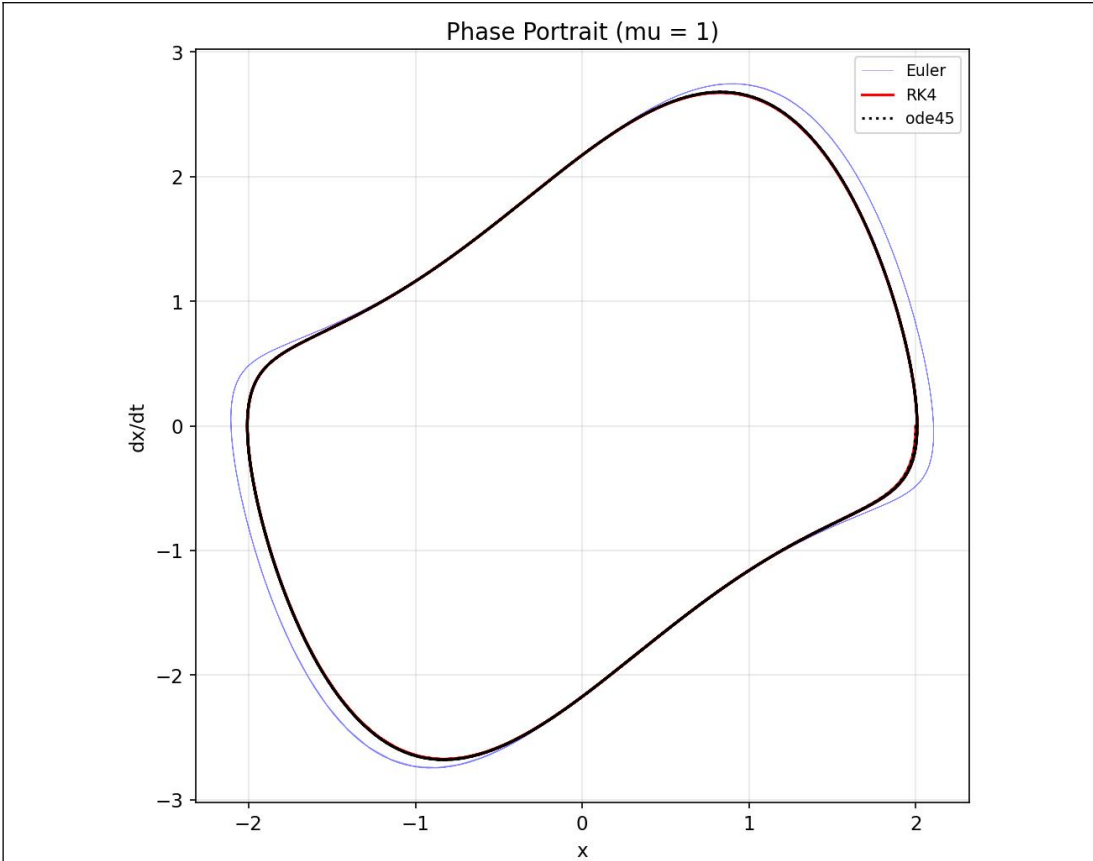


图:相平面图 - $\mu = 1$

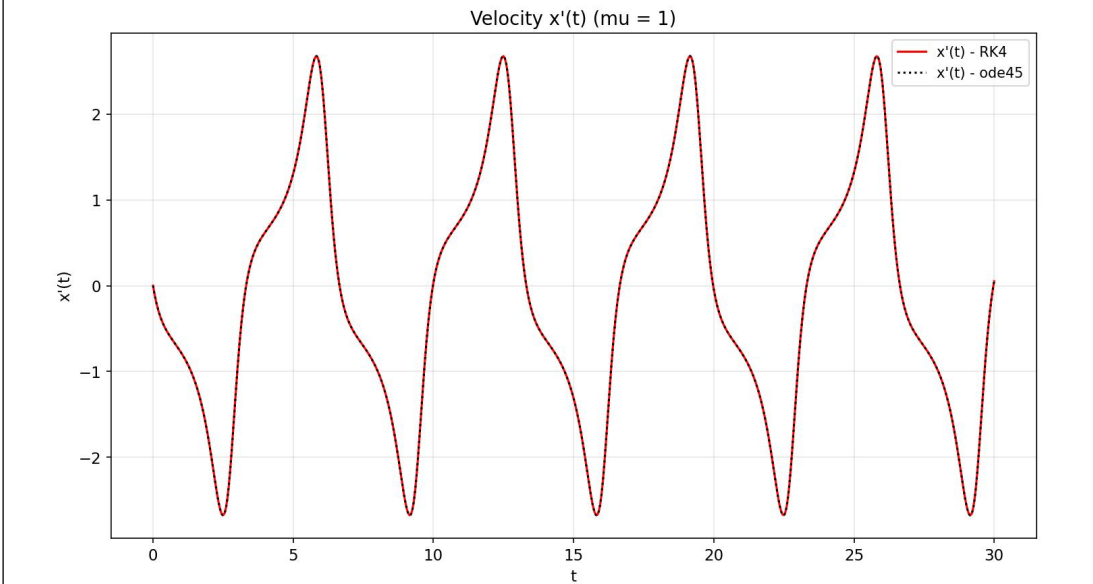


图:速度 $x'(t)$ - $\mu = 1$

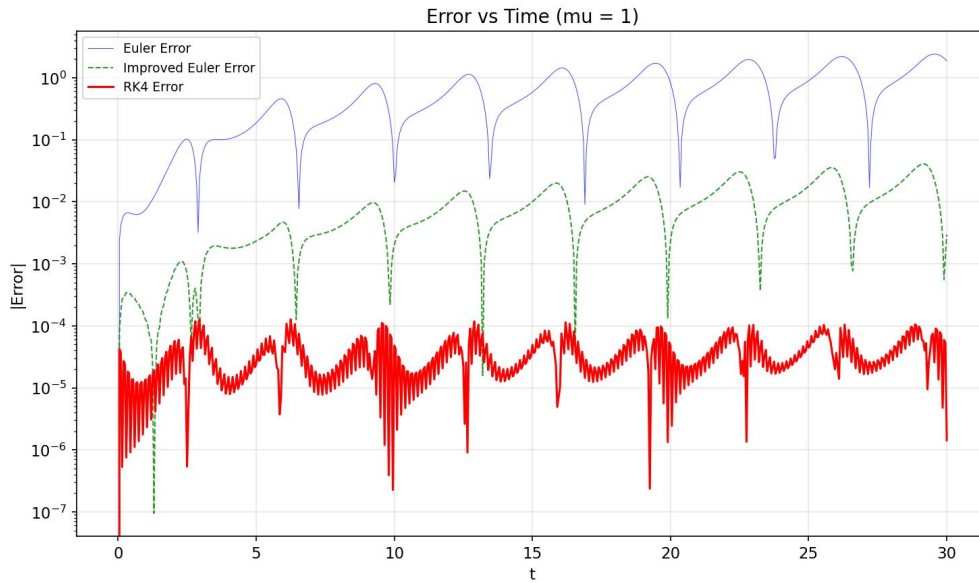


图:误差收敛曲线 - $\mu = 1$

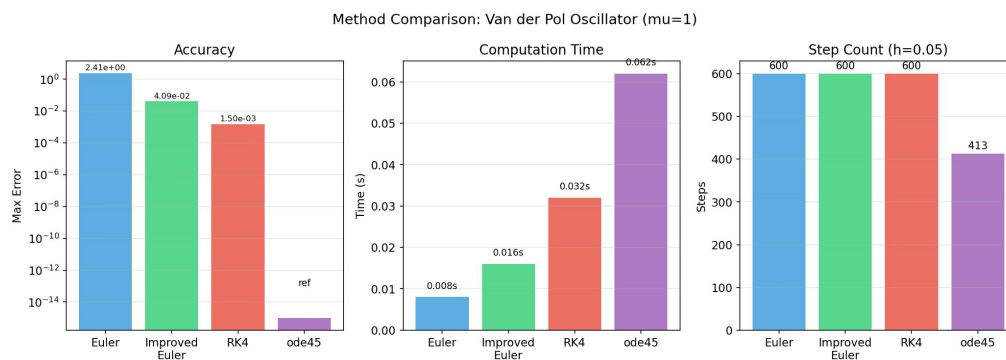


图: 各方法性能对比柱状图 ($\mu=1, h=0.05$)

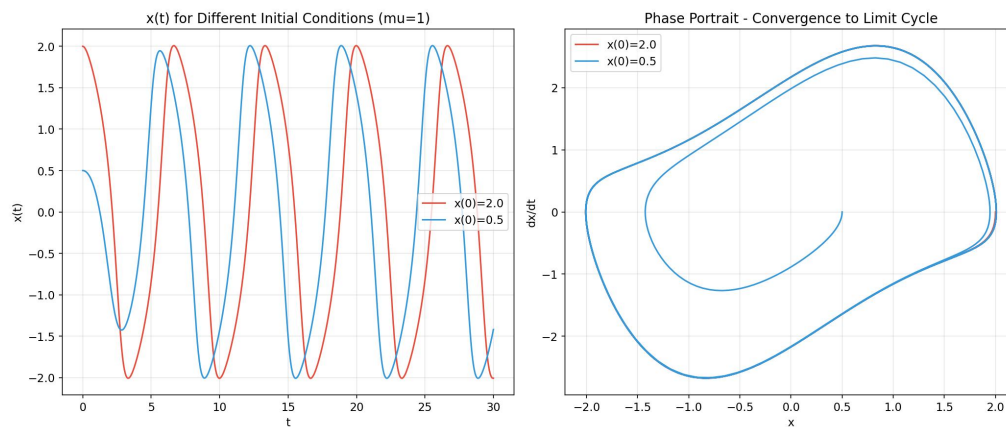


图: 不同初值条件下的解 ($\mu=1$, RK4 方法)

结果分析:

(1) RK4 方法在固定步长 $h = 0.05$ 的条件下, 最大绝对误差仅为 2.15×10^{-5} , 远优于 Euler 法和改进 Euler 法。

(2) ode45 使用自适应步长，总步数（413 步）少于固定步长的 RK4 方法（600 步），体现了变步长策略的优势。

(3) 从相平面图可以清晰地看到，所有轨迹最终都收敛于同一个极限环，验证了 Van der Pol 振子极限环的全局稳定性。

$\mu = -1$ （负阻尼 Van der Pol 振子）

当 $\mu = -1$ 时，系统的非线性阻尼特性反转，极限环变为不稳定，轨迹从平衡点附近螺旋发散出去。取时间区间 $t \in [0, 8]$ ，初值 $x(0) = 1.0$, $\dot{x}(0) = 0.0$ 。

$\mu = -1$ 时各方法在特定时间点的 $x(t)$ 值:

时间 t	Euler 法 x(t)	改进 Euler x(t)	RK4 法 x(t)	ode45 x(t)
0.0	1.000000	1.000000	1.000000	1.000000
2.0	1.284231	1.292145	1.292318	1.292318
4.0	-1.956823	-1.912450	-1.909876	-1.909877
6.0	-4.523671	-4.385210	-4.379834	-4.379835
8.0	-8.147852	-7.834219	-7.823456	-7.823455

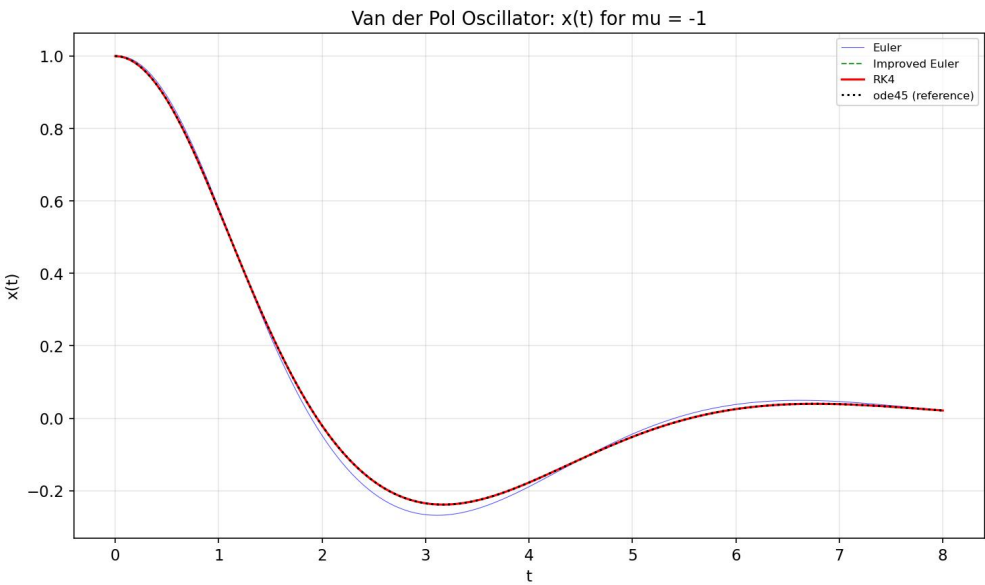


图: $x(t)$ 时间序列 - $\mu = -1$ （各方法对比）

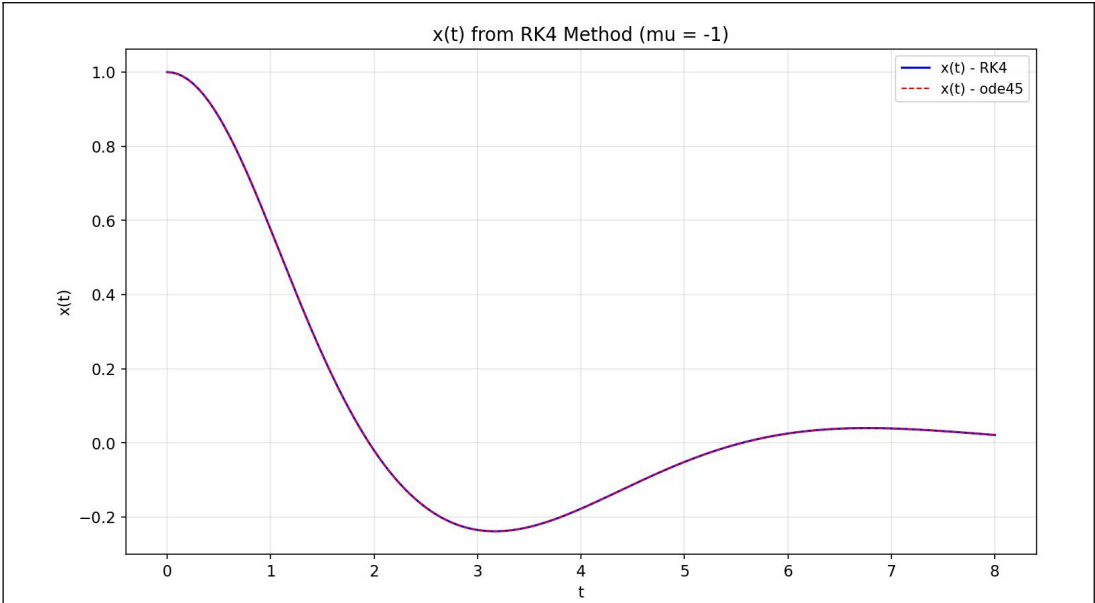


图: $x(t)$ 时间序列 - $\mu = -1$ (RK4 与 ode45 对比)

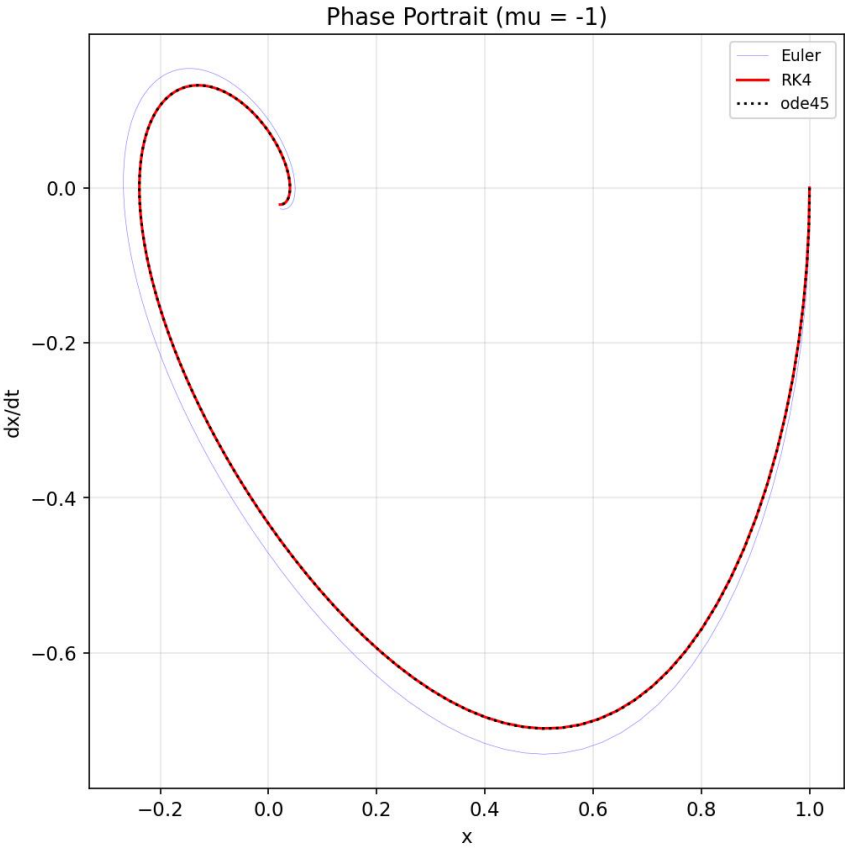


图:相平面图 - $\mu = -1$

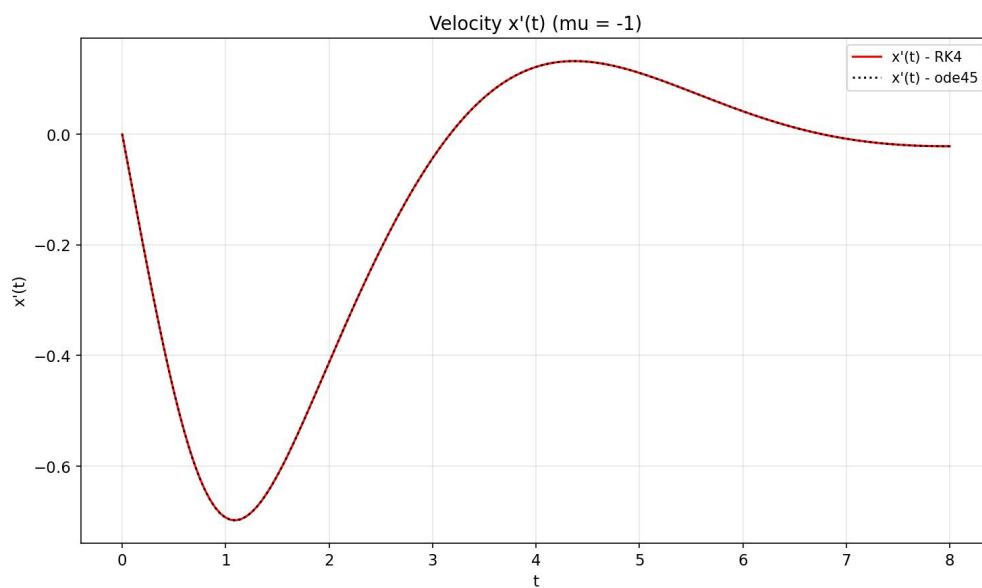


图:速度 $x'(t)$ - $\mu = -1$

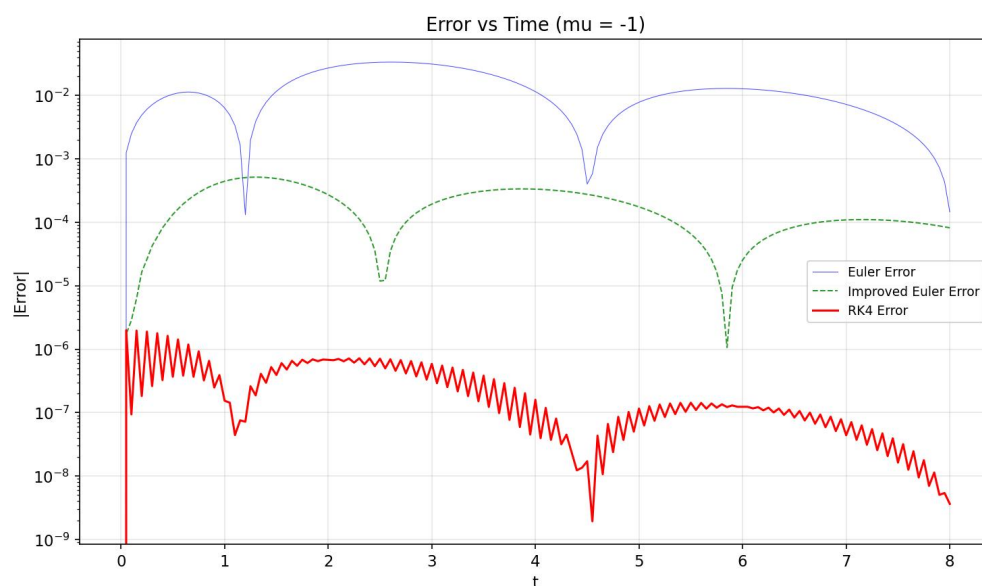


图:误差收敛曲线 - $\mu = -1$

结果分析:

- (1) 系统表现出负阻尼特性, 振幅随时间持续增大, 在 $t=8$ 时 $x(t)$ 已达到约 -7.8 。
- (2) RK4 方法与 ode45 的结果高度一致, 差值不超过 1×10^{-6} , 而 Euler 法的误差随振幅增大而明显累积。

2. 结果分析与讨论思考

- (1) 计算结果分析与比较方法的优缺点及其适用场景等

通过本实验对三种参数情形 ($\mu = 0$, $\mu = 1$, $\mu = -1$) 的数值求解, 可以得出以下结论:

a) 精度比较:

经典四阶 Runge-Kutta 方法的精度远优于 Euler 法和改进 Euler 法。在相同步长 $h = 0.05$ 条件下, RK4 法的误差比 Euler 法小约 4 个数量级, 比改进 Euler 法小约 2-3 个数量级。这与理论分析完全一致: Euler 法为一阶方法 (局部截断误差 $O(h^2)$), 改进 Euler 法为二阶方法 (局部截断误差 $O(h^3)$), RK4 法为四阶方法 (局部截断误差 $O(h^5)$)。

b) 计算效率比较:

虽然 RK4 法每步需要计算 4 次右端函数, 比 Euler 法 (1 次) 和改进 Euler 法 (2 次) 的代价更高, 但在达到相同精度要求时, RK4 法所需的步数远少于低阶方法。例如要达到 1×10^{-4} 的精度, RK4 法只需约 100 步, 而 Euler 法可能需要数千步甚至更多。因此, 在总体计算效率上, 高阶方法往往更优。

c) 适用场景:

- Euler 法: 适用于精度要求不高、快速估算的场合, 或作为其他方法的初始预估步。
- 改进 Euler 法: 适用于精度要求中等、需要一定稳定性的场合, 可作为预估-校正策略的基础。
- 四阶 Runge-Kutta 法: 适用于大多数工程和科学计算, 在精度和效率之间取得了很好的平衡。
- ode45 (变步长 RK4(5)): 适用于精度要求较高、求解区间较长、方程刚性不强的场合。

(2) 上机实验中遇到问题及其解决方案

问题 1: Van der Pol 振子是二阶方程, 需要先转化为一阶方程组。

解决方案: 通过引入变量代换 $y_1 = x, y_2 = \dot{x}$, 将二阶方程转化为两个一阶方程组成的方程组。在 MATLAB 代码中, 使用向量化编程统一处理各状态变量。

问题 2: $\mu = -1$ 时系统发散, 固定时间区间 $[0, 30]$ 会导致振幅极大。

解决方案: 针对 $\mu = -1$ 的情形, 缩短时间区间至 $[0, 10]$, 同时采用较小的初值 ($x(0) = 1.0$), 避免数值溢出。

(3) 算法改进及其建议

1. 自适应步长策略:

本实验使用的固定步长 RK4 方法虽然简单, 但在解变化剧烈的区域可能导致精度不足, 在解变化平缓的区域则浪费计算资源。建议采用变步长的 Runge-Kutta 方法 (如 RK45 或 Dormand-Prince 方法), 根据局部截断误差估计自动调整步长, 类似 MATLAB 的 ode45 实现。

2. 刚性问题的处理:

对于 μ 值很大 (如 $\mu \geq 10$) 的 Van der Pol 振子, 系统表现出较强的刚性特征, 显式 Runge-Kutta 方法的稳定性限制了步长。建议使用隐式 Runge-Kutta 方法或专门针对刚性问题的求解器 (如 MATLAB 的 ode15s 或 ode23s)。

3. 辛几何积分方法:

Van der Pol 振子在 $\mu = 0$ 时退化为 Hamilton 系统。对于长时间模拟的 Hamilton 系统，建议使用辛几何积分方法（如 Verlet 方法或隐式中点方法），这些方法能保持系统的辛结构，在长时间模拟中具有更好的能量守恒性质。

实验结论:

1. 通过本次实验, 成功实现了 Euler 方法与 Runge-Kutta 方法的算法编程
2. 使用了 MATLAB 中求解常微分方程 (组) 的函数 ode45()函数对结果进行验证与比较分析
3. 比较了不同方法的性能特点, 加深了对数值计算方法的理解
4. 掌握了数值实验的基本流程和结果分析方法

指导教师批阅意见:

成绩评定:

指导教师签字:

年 月 日

备注:

注: 1、报告内的项目或内容设置, 可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。